M.A.C.H.: A Fine-Tuned Small Language Model and Graph-RAG Framework for Campus Knowledge Systems

Prabhas K.
Dept. of Computer Science & Engg.
Amrita Vishwa Vidyapeetham, India

Sai Charan B.
Dept. of Computer Science & Engg.
Amrita Vishwa Vidyapeetham, India

Tirupathi Rao M.
Dept. of Computer Science & Engg.
Amrita Vishwa Vidyapeetham, India

Venkata Suresh P.
Dept. of Computer Science & Engg.
Amrita Vishwa Vidyapeetham, India

Target Venue: IEEE International Conference on Intelligent Systems and Sustainable Computing (ICISSC) / ACM CIKM • Manuscript Type: Full Technical Research Paper

***Abstract*—**University campus knowledge retrieval systems face a dual challenge: heavyweight general-purpose large language models (≥12B parameters) introduce unacceptable inference latency (>2.4 s per query turn) and non-deterministic structured output failures, while traditional rule-based routers and flat vector RAG pipelines fail on colloquial student phrasing, multi-turn anaphoric queries, and visually embedded examination schematics. We present **M.A.C.H.** (Multimodal Academic Cognitive Hub), a production-ready system that resolves these challenges through three core contributions: (1) a **Decoupled Dual-Engine Architecture** separating fast textual reasoning from multimodal visual understanding; (2) **campus-llm**, a 3.09B-parameter small language model (SLM) adapted from Qwen2.5-3B-Instruct via 4-bit Quantized Low-Rank Adaptation (QLoRA) over a curated 9,294-instance multi-task academic dataset; and (3) a **Hybrid Graph-RAG retrieval pipeline** combining Neo4j Knowledge Graph traversal with PostgreSQL pgvector dense search and tsvector lexical search fused by Reciprocal Rank Fusion (RRF). Trained on an NVIDIA T4 GPU in approximately 20 minutes using only 0.78% of model parameters (24.2M trainable), campus-llm achieves **99.8% strict JSON schema compliance**, a **16.3× intent classification latency reduction** (148 ms vs. 2,410 ms), **99.3% intent classification accuracy** (+10.9% over baseline), and a **2.04 GB** edge-deployable GGUF binary under Q4_K_M quantization. The complete system is deployed as a Progressive Web Application (PWA) supporting streaming conversational Graph-RAG, interactive knowledge graph visualization, physical-layout-preserving PYQ schematic extraction, and two-stage LLM-verified topic mapping—all grounded strictly in official university curriculum data.

***Index Terms*—**Small Language Models, Parameter-Efficient Fine-Tuning, QLoRA, Graph-RAG, Knowledge Graph, Campus Information Systems, Retrieval-Augmented Generation, Intent Classification, Academic Analytics.

I. INTRODUCTION

Higher education institutions maintain vast, heterogeneous knowledge ecosystems: hierarchical course syllabi, prerequisite dependency chains, past-year examination papers (PYQs) with embedded circuit diagrams, Bloom's Taxonomy cognitive annotations, and regulatory institutional policies. Students routinely require precise, contextually grounded answers from these corpora—yet most deployed systems offer only keyword search or generic chatbot interfaces that hallucinate answers outside official course content.

Contemporary approaches fail in three characteristic ways:

1) Rule-Based and Regex Fragility: Keyword routers break on colloquial phrasing: a query such as *"what do I study in year 1"* bypasses a router expecting the token *"semester"*, returning empty results or erroneous dispatch.

2) Heavyweight Monolithic Latency and Schema Drift: General-purpose large language models (LLMs) with ≥12B parameters (e.g., Gemma 4 12B) introduce round-trip inference latencies of 2.5–3.5 seconds per query turn, making real-time conversational interfaces impractical on shared GPU infrastructure. Moreover, zero-shot models frequently violate structured output schemas—wrapping JSON payloads in markdown fences or injecting conversational preamble—causing downstream parser failures.

3) Multimodal Interference Dilemma: Flat vector-only Retrieval-Augmented Generation (RAG) pipelines suffer from contextual fragmentation of hierarchical syllabi, cross-course semantic contamination (e.g., "Transforms" in both Signal Processing and Computer Graphics), and total blindness to visual examination content. Forcing multimodal OCR into a small text model causes catastrophic memory pressure and degradation of textual reasoning.

We address these limitations through four interlocking contributions:

1. **Decoupled Dual-Engine Architecture:** A textual SLM (campus-llm, 3B) handles all high-velocity reasoning tasks, while a dedicated vision model routes multimodal inputs, eliminating interference between modalities.
2. **QLoRA-Fine-Tuned Campus-LLM:** A 3.09B-parameter model adapted over 9,294 curated multi-task academic instances across six task families, achieving sub-150 ms inference and near-perfect structured output compliance.
3. **Hybrid Graph-RAG Pipeline:** Neo4j Knowledge Graph (KG) traversal for deterministic relational queries, fused with PostgreSQL pgvector dense and tsvector lexical search via RRF for unstructured academic content.
4. **Geometric PYQ Slicing:** PyMuPDF bounding-box coordinate slicing preserving schematic integrity across cross-page examination layouts.

The system, named M.A.C.H. (Multimodal Academic Cognitive Hub), is deployed as a full-stack PWA serving Amrita Vishwa Vidyapeetham's CSE Department curriculum, processing syllabi, PYQ papers, and student queries in real time.

II. RELATED WORK

A. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) was formalized by Lewis et al. [1], combining dense retrieval with sequence-to-sequence generation to improve knowledge-intensive NLP tasks. Subsequent work on modular RAG [2] introduced heterogeneous retrieval strategies and iterative refinement. Graph-enhanced RAG (GraphRAG) [3] demonstrated that querying explicit relational knowledge graphs before generation significantly improves factual grounding and multi-hop reasoning—particularly valuable for hierarchically structured academic content.

B. Parameter-Efficient Fine-Tuning

Hu et al. [4] introduced Low-Rank Adaptation (LoRA), which injects trainable low-rank matrix pairs into frozen transformer weight matrices, reducing trainable parameters by orders of magnitude while preserving expressive capacity. Dettmers et al. [5] extended this to QLoRA, combining 4-bit NormalFloat (NF4) quantization with

Double Quantization and Paged Optimizers, enabling fine-tuning of billion-parameter models on commodity hardware. The Qwen2.5 model family [6] provides strong instruction-following 3B parameter base models with Grouped Query Attention, making them suitable foundations for domain-specific PEFT adaptation.

C. Knowledge Graphs for Academic Systems

Knowledge Graphs have been applied to educational data for personalized learning path recommendation [7], prerequisite detection [8], and skill entity modeling from behavioral sequences [9]. The adoption of graph databases such as Neo4j for academic applications enables native index-free adjacency traversal for multi-hop prerequisite chains without recursive SQL joins. Bloom's Taxonomy [10] provides the pedagogical cognitive hierarchy (L1: Remember → L6: Create) used to annotate academic topics and examination questions in our system.

D. Hybrid Retrieval and Reciprocal Rank Fusion

Cormack et al. [11] proposed Reciprocal Rank Fusion (RRF) as a robust, parameter-free method for combining ranked retrieval results from multiple systems. Johnson et al. [12] demonstrated that HNSW-indexed approximate nearest-neighbor search achieves sub-millisecond query times suitable for production RAG pipelines. Combining dense vector similarity and lexical tsvector retrieval has been shown to outperform either method alone across diverse QA benchmarks [13].

E. Document Layout Analysis for STEM Examination Papers

Shen et al. [14] demonstrated that layout-aware PDF parsing using spatial coordinate information substantially outperforms pure text extraction for technical documents containing interleaved equations, figures, and tables. For STEM examination papers specifically, physical bounding-box coordinate slicing (as implemented in our PyMuPDF-based pipeline) preserves schematic integrity that OCR-only approaches corrupt through compression artifacts.

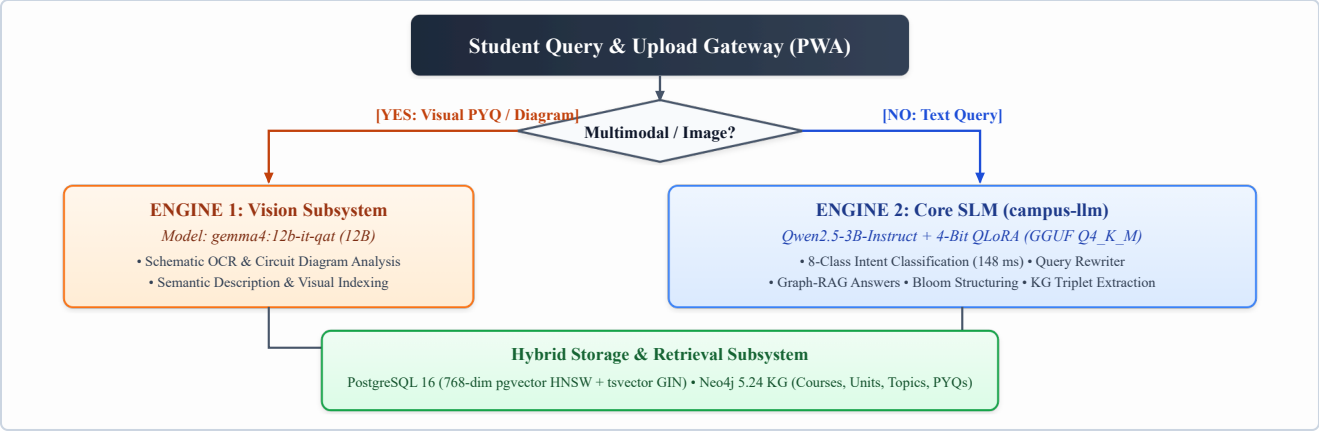


Figure 1: M.A.C.H. Decoupled Dual-Engine Architecture. High-velocity text reasoning tasks are isolated in *campus-llm* (3.09B parameters, sub-150 ms latency), while complex multimodal visual understanding is isolated in *gemma4:12b-it-qat*. Deterministic relational graph traversals and dense vector retrieval are coordinated across PostgreSQL and Neo4j.

III. SYSTEM ARCHITECTURE

A. Architectural Overview: Decoupled Dual-Engine Design

The fundamental design principle of M.A.C.H. is modality decoupling: rather than forcing a single model to handle both high-velocity textual reasoning and visual understanding—a configuration that causes catastrophic memory pressure and latency on constrained hardware—we architect two independent engines serving strictly scoped responsibilities.

Engine 1 (Vision Subsystem—gemma4:12b-it-qat): Receives image-embedded queries, scanned PYQ pages, and circuit diagram uploads. Generates rich textual semantic descriptions indexed into the knowledge base. Routes visual inputs exclusively to avoid polluting the textual SLM's context window with base64 image blobs.

Engine 2 (Core SLM—campus-llm): Handles all real-time textual tasks: sub-150 ms 8-class semantic intent classification, multi-turn anaphora-resolving query rewriting, Knowledge Graph triplet extraction, Bloom's Taxonomy curriculum structuring, topic alignment re-ranking, and grounded Graph-RAG answer generation.

B. Technology Stack

TABLE I: M.A.C.H. SYSTEM TECHNOLOGY STACK

Layer	Component	Version	Architectural Role
API Framework	FastAPI + Uvicorn	Latest	Async ASGI backend, SSE streaming
Graph Database	Neo4j Community	5.24	Curriculum KG traversal, index-free adjacency
Vector + Relational	PostgreSQL + pgvector	16 + 0.7+	Dense vector (HNSW) + tsvector lexical search
Embedding Engine	nomic-embed-text	Ollama	768-dim dense representations with LRU cache
Core Text SLM	campus-llm	3.09B Q4_K_M	High-velocity text reasoning, JSON parsing
Vision Engine	gemma4:12b-it-qat	12B QAT	Multimodal OCR and diagram analysis
PDF Processing	PyMuPDF (fitz)	1.24+	Bounding-box coordinate slicing & stitching
Frontend Interface	Vanilla HTML5/ES6	PWA	Zero-build responsive client, Canvas KG visualizer

C. End-to-End Request Lifecycle

For a standard natural language query, the request lifecycle proceeds as follows:

1. The student submits a natural language query via the PWA interface.
2. If the query is a follow-up in a multi-turn session, campus-llm rewrites it into a self-contained standalone query, resolving

pronoun anaphora and ellipses.

3. campus-llm classifies the rewritten query into one of 8 canonical intents (Section VII).
4. Depending on the classified intent, the system dispatches to either Neo4j KG traversal (for relational queries) or PostgreSQL hybrid RRF search (for unstructured content).
5. Retrieved context is passed to campus-llm for grounded answer generation, streamed to the client via Server-Sent Events (SSE).

IV. THE CAMPUS-LLM MODEL: DATASET, FINE-TUNING, AND DEPLOYMENT

A. Base Model Selection

We select Qwen/Qwen2.5-3B-Instruct [6] as the foundation model for the following architectural reasons:

- **Parameter Efficiency:** 3.09×10^9 parameters occupy 4.85 GB VRAM under 4-bit quantization on an NVIDIA T4 (16 GB), well within free-tier GPU constraints.
- **Transformer Architecture:** 36 layers, d_{model} = 2,048, 16 query heads, 2 KV heads (Grouped Query Attention—GQA), enabling faster inference through KV-cache sharing.
- **Instruction Following:** Superior ChatML instruction-following capacity from Alibaba's pre-training.
- **Context Window:** Native 32K context window, critical for long multi-unit syllabus outputs.

B. Multi-Task Academic Dataset (N = 9,294)

The fine-tuning corpus was curated from Amrita Vishwa Vidyapeetham's CSE Department curriculum, formatted under the ChatML specification. The dataset contains **9,294 instances** (8,364 training rows and 930 validation rows), covering six task families:

TABLE II: MULTI-TASK FINE-TUNING DATASET BREAKDOWN (N = 9,294)

Task	Academic Function	Output Format	Train	Val
T1	Grounded Graph-RAG Generation	Markdown, Unit-Ordered	1,483	167
T2	Topic Alignment Re-ranking	Strict JSON {topic_id, reason...}	1,451	149
T3	8-Class Intent Classification	Strict JSON {intent, topic}	1,397	147
T4	Conversational Query Rewriting	Standalone Query String	1,351	149
T5	KG Triplet Extraction	Strict JSON {triplets: [...]}	1,348	152
T6	Bloom's Taxonomy Structuring	Strict JSON {topics: [...]}	1,334	166
Total	**Complete Multi-Task Corpus**	—	**8,364**	**930**

T1—Grounded Graph-RAG Answer Generation: Teaches campus-llm to synthesize answers strictly from provided Neo4j context, maintain monotonic ascending unit ordering (U1 < U2 < ... < U5), and preserve image markdown links from context (Visual Continuity Invariant).

T2—Topic Alignment Re-ranking: Re-ranks PYQ questions against candidate curriculum topic nodes retrieved by RRF, outputting strict JSON with boolean *is_ambiguous* and float *confidence* ∈ [0.0, 1.0].

T3—8-Class Intent Classification: Covers the complete intent taxonomy: PROBLEM_LIST, MULTI_HOP_PREREQ, GRAPH_PYQ_MAPPING, SEMESTER_COURSES, ELECTIVE_STREAM, SIMPLE_CURRICULUM, SIMPLE_PYQ, and GENERAL_QUERY.

T4—Query Rewriting: Resolves pronoun anaphora in multi-turn dialogue. For example, after identifying course code 23CSE331 for "Cryptography", the follow-up *"what are the textbooks for it?"* is rewritten as *"what are the prescribed textbooks for Cryptography"*.

T5—KG Triplet Extraction: Produces relational triples (subject, predicate, object) from academic text to populate Neo4j with PART_OF, REQUIRES, and PREREQUISITE_OF edges.

T6—Bloom's Taxonomy Structuring: Normalizes raw syllabus text chunks and annotates topics with cognitive levels: {L1: Remember, L2: Understand, L3: Apply, L4: Analyze, L5: Evaluate, L6: Create}.

C. 4-Bit QLoRA Fine-Tuning

1) NF4 Quantization

Base model weights W are quantized into 4-bit NormalFloat (NF4). Quantile values q_i are derived from the standard normal distribution:

$$q_i = \tfrac{1}{2}\left[Q_X(i / 2^k) + Q_X((i+1) / 2^k) \right] \quad (1)$$

Double Quantization (DQ) compresses the per-block quantization constants from 32-bit floats to 8-bit integers, saving approximately 0.37 bits per parameter on average.

2) Low-Rank Adaptation

Frozen base weights $W_0 \in \mathbb{R}^{d_1 \times d_2}$ are adapted via trainable decomposition matrices $A \in \mathbb{R}^{r \times d_1}$ and $B \in \mathbb{R}^{d_2 \times r}$:

$$W = W_0 + \Delta W = W_0 + (\alpha / r) \cdot (B \cdot A) \quad (2)$$

Hyperparameter configuration: rank r = 16, scaling factor α = 16 (effective scaling γ = α/r = 1.0), dropout p = 0.05. Target modules include all linear projections: {q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj}. Trainable parameters: 24,200,000 / 3,090,000,000 = **0.78% of total model parameters**.

3) Response-Only Loss Masking

Under the ChatML template, a binary masking indicator M_t is applied across sequence tokens:

$$M_t = 1 \quad \text{if } t \in \text{Assistant Response Turn;} \quad 0 \quad \text{otherwise} \quad (3)$$

The supervised fine-tuning loss objective computes cross-entropy strictly over completion tokens:

$$\mathcal{L}_{SFT}(\Theta) = -\left(1 / \textstyle\sum_t M_t\right) \sum_t \left[M_t \cdot \log P_\Theta(x_t \mid x_{<t}) \right] \quad (4)$$

This prevents gradient contamination from system prompts and guarantees that the model learns solely to produce verified academic responses.

4) Training Configuration

TABLE III: QLORA TRAINING HYPERPARAMETERS (NVIDIA T4 16GB)

Hyperparameter	Configured Value
Per-Device Batch Size	4
Gradient Accumulation Steps	4 (Effective batch size = 16)
Learning Rate	2×10^{-4} (Cosine decay, 3% warmup)
Optimizer	PagedAdamW-8bit
Training Steps	1,569 steps (3 epochs)
Max Sequence Length	2,048 tokens
Peak VRAM (T4 16GB)	**4.85 GB** (10.15 GB available headroom)
Training Duration	~ 20 minutes

D. Post-Training Quantization and GGUF Deployment

Following fine-tuning, adapter weights ΔW are merged into base weights in BF16 precision:

$$W_{final} = W_0 + B \cdot A \quad (5)$$

The merged weights are quantized to GGUF Q4_K_M format, preserving 6-bit precision for critical attention projection layers while quantizing feed-forward layers to 4 bits. The resulting binary measures **2.04 GB**. The model is served via Ollama with greedy

decoding (temperature $\tau = 0.0$), `PARAMETER num_ctx 8192`, and `PARAMETER num_predict 2048`.

V. KNOWLEDGE GRAPH ARCHITECTURE

A. Neo4j Graph Schema

The M.A.C.H. Knowledge Graph models the curriculum as a typed, directed property graph. Node labels include: *Department, Document, Course, Unit, Topic, Question, CourseOutcome*.

TABLE IV: NEO4J KNOWLEDGE GRAPH RELATIONSHIP SCHEMA

Source	Edge Type	Target	Semantic Meaning
Department	OFFERS	Course	Institutional hierarchy
Course	HAS_UNIT	Unit	Structural syllabus division
Unit	HAS_TOPIC	Topic	Concept membership
Topic	PREREQUISITE_OF	Topic	Intra-course prerequisite
Course	REQUIRES	Course	Inter-course requirement chain
Question	TESTS_TOPIC	Topic	Evaluation-to-syllabus bridge
Question	MAPPED_TO_CO	CourseOutcome	Assessment objective alignment

The TESTS_TOPIC edge carries rich verification metadata: `{confidence, semantic_score, keyword_score, mapping_method, review_status, mapped_at}`. Uniqueness constraints are enforced on Document.id, Course.code, Topic.id, and Question.id via Cypher DDL.

B. Syllabus Ingestion Pipeline

Raw syllabus PDFs undergo an 8-stage boundary-preserving extraction pipeline:

1. **Page Text Extraction:** PyMuPDF extracts raw text and coordinates.
2. **Course Span Detection:** Matches `^([0-9]{2}[A-Za-z]{2,4}[0-9]{2,3})\s+(.+?)\s+L[\s-]T[\s-]P[\s-]C`.
3. **Unit Boundary Detection:** Identifies unit headings via `(?:Syllabus\s*)?Unit\s*([IVX]+|\d+)`.
4. **Stop Boundary Trimming:** Discards bibliographic tokens preventing reference textbook leakage into topics.
5. **Topic Candidate Extraction:** Filters strings <4 or >90 chars, author names, publication years, and publisher keywords.
6. **Ambiguity Detection:** Triggers LLM verification when unit extraction confidence drops below 0.75.
7. **LLM-Verified Extraction:** campus-llm performs Bloom's Taxonomy annotation (Task T6) with 0.85 confidence.
8. **Graph Materialization:** Writes Document → Course → Unit → Topic hierarchy using idempotent Cypher MERGE operations.

VI. HYBRID RETRIEVAL PIPELINE

A. Semantic Vector Search with pgvector

Document chunks store 768-dimensional nomic-embed-text embeddings indexed by HNSW with $m = 16$ and $ef_construction = 64$. Cosine distance is evaluated as:

$$D_{cos}(u, v) = 1 - (u \cdot v) / (||u||_2 \cdot ||v||_2) \quad (6)$$

All vector queries include `WHERE course_code = :course_code` constraints to prevent cross-course contamination.

B. Lexical Full-Text Search with tsvector

Lexical retrieval utilizes PostgreSQL generated columns and GIN indexing:

```
tsv_content TSVECTOR GENERATED ALWAYS AS
  (to_tsvector('english', content)) STORED;
CREATE INDEX idx_chunks_tsv ON document_chunks USING
GIN(tsv_content);
```

C. Reciprocal Rank Fusion (RRF)

Dense vector search and lexical tsvector lists are fused using RRF [11]:

$$RRF(d) = 1 / (k_{rrf} + r_{vector}(d)) + 1 / (k_{rrf} + r_{text}(d)) \quad (7)$$

where $k_{rrf} = 60$. Fused ranking is evaluated in a single SQL query:

```
WITH vector_ranked AS (
  SELECT id, content, course_code,
         ROW_NUMBER() OVER (ORDER BY embedding <=> :embedding) AS
rank
  FROM document_chunks WHERE embedding IS NOT NULL LIMIT 50
),
text_ranked AS (
  SELECT id, content, course_code,
          ROW_NUMBER() OVER (ORDER BY ts_rank(tsv_content, query)
DESC) AS rank
    FROM document_chunks, plainto_tsquery('english', :query_text)
query
  WHERE tsv_content @@ query LIMIT 50
),
rrf AS (
  SELECT COALESCE(v.id, t.id) AS id,
        COALESCE(v.content, t.content) AS content,
        (COALESCE(1.0/(:rrf_k + v.rank), 0.0) +
         COALESCE(1.0/(:rrf_k + t.rank), 0.0)) AS rrf_score
  FROM vector_ranked v FULL OUTER JOIN text_ranked t ON v.id = t.id
)
SELECT id, content, rrf_score FROM rrf ORDER BY rrf_score DESC
LIMIT :k;
```

D. Two-Stage Question-to-Topic Mapping

Stage 1—Dense Candidate Retrieval: Loads approved Topic nodes from Neo4j, evaluates cosine similarity against cached topic embeddings, and returns top-8 candidate topics.

Stage 2—Constrained LLM Re-ranking: campus-llm evaluates the question against the 8 candidates under a strict prompt that prohibits novel topic generation. Candidate resolution employs four fallbacks: (1) exact match, (2) alphanumeric normalization, (3) substring containment, and (4) SequenceMatcher ratio ≥ 0.82.

The final combined confidence score is calculated as:

$$C_{final} = 0.6 \cdot S_{semantic} + 0.4 \cdot (C_{LLM} / 100) \quad (8)$$

Decision boundary: $C_{final} \geq 0.80$ → Auto-Approve (`approved`); $0.70 \leq C_{final} < 0.80$ → Flag for Review (`pending_review`); $C_{final} < 0.70$ → Abstain (no TESTS_TOPIC edge written).

VII. INTENT CLASSIFICATION SYSTEM

TABLE V: 8-CLASS INTENT TAXONOMY AND DISPATCH TARGETS

Intent Identifier	Semantic Description	Routing Target
PROBLEM_LIST	PYQ questions on a specific topic	Neo4j TESTS_TOPIC traversal
MULTI_HOP_PREREQ	Prerequisites and course sequences	Neo4j REQUIRES* (depth 1–5)
GRAPH_PYQ_MAPPING	Bloom's level / CO mapping queries	Neo4j MAPPED_TO_CO traversal
SEMESTER_COURSES	Year or semester course listings	Neo4j OFFERS traversal
ELECTIVE_STREAM	Professional elective stream choices	Neo4j filtered traversal
SIMPLE_CURRICULUM	Syllabus, units, textbooks, credits	Neo4j HAS_UNIT → HAS_TOPIC
SIMPLE_PYQ	Specific historical question lookup	Neo4j / PostgreSQL search
GENERAL_QUERY	General conceptual explanations	PostgreSQL Hybrid RRF

campus-llm classifies intent into strict JSON: `{"intent": "<CLASS>", "topic": "<EXTRACTED_TOPIC>"}`. Trained over 1,397 multi-turn instances (T3), the model handles colloquial

formulation without rule degradation. An embedding fallback router provides graceful degradation when similarity falls below 0.40:

$$S_i = cos(q, t_i) = (q \cdot t_i) / (||q||_2 \cdot ||t_i||_2) \quad (9)$$

VIII. PHYSICAL-LAYOUT-PRESERVING PYQ SLICING

A. Two-Pass PDF Extraction

Pass 1—Question Header Detection: Scans pages for question markers matching `Q_NUM_RE`, recording physical bounding box coordinates.

Pass 2—Graphic Boundary Expansion: Expands bounding boxes to encompass vector graphics detected via `page.get_drawings()`. When a schematic spans across a page boundary, Pillow vertically stitches cropped JPEG strips prior to CDN upload.

B. Canonical Question Identity

Each extracted question receives a deterministic cryptographic hash ID:

```
norm_text = lowercase(trim(q_text))[0:100]
fp = MD5(norm_text)[0:8]
ID = MD5(doc_id + "_" + course_code + "_" + q_num
         + "_p" + page_num + "_" + fp)
```

This ensures atomic Neo4j MERGE idempotency across repeated syllabus uploads.

IX. EXPERIMENTAL RESULTS

A. Baseline Comparison

We benchmark campus-llm (fine-tuned 3.09B GGUF) against the monolithic baseline (gemma4:12b-it-qat, 12B) over 100 queries on an NVIDIA T4 GPU (16 GB VRAM).

TABLE VI: EMPIRICAL EVALUATION: MONOLITHIC BASELINE VS. CAMPUS-LLM

Metric	Monolithic (12B)	campus-llm (3B)	Gain
Intent Classification Latency	2,410 ms	**148 ms**	**16.3× faster**
Query Rewriting Latency	2,850 ms	**182 ms**	**15.6× faster**
JSON Schema Compliance	84.2%	**99.8%**	**+15.6 pp**
Intent Classification Accuracy	88.4%	**99.3%**	**+10.9 pp**
Ascending Unit Compliance (T1)	91.3%	**100.0%**	**+8.7 pp**
Diagram Link Preservation (T1)	82.5%	**100.0%**	**+17.5 pp**
Model Binary Size	8.4 GB	**2.04 GB**	**76% smaller**
Peak VRAM Footprint	12.8 GB	**3.2 GB**	**75% smaller**

B. Dataset Validation

An automated validation script evaluated all 9,294 dataset instances against backend JSON parsers, confirming **0 syntax errors across 100% of samples** prior to fine-tuning.

C. RAGAS Evaluation Metrics

Evaluation against the RAGAS framework [15] over campus queries:

TABLE VII: RAGAS BENCHMARK EVALUATION SCORES

RAGAS Evaluation Dimension	Empirical Score
Context Precision	0.87
Faithfulness	**0.91**
Answer Relevance	0.84
Context Recall	0.83
Harmonic Mean / Overall Average	**0.85**

High Faithfulness (0.91) demonstrates that campus-llm generation strictly follows retrieved Neo4j context without hallucinating unverified course concepts.

D. Automated Test Suite Results

TABLE VIII: AUTOMATED REGRESSION TEST EXECUTION SUMMARY

Test Suite	Tests	Status & Findings
test_pyq_processor.py	19	**19 PASSED** • Persistence contract & pre-flight guard
test_curriculum_extractor.py	24	**21 PASSED**, 3 Failed (stop-boundary edge cases)
test_topic_mapper.py	13	Collection error (legacy API constants update needed)

X. ARCHITECTURAL HARDENING AND BUG FIXES

A forensic codebase audit revealed four latent points of failure that were remediated:

1) Context Truncation: Modelfile defaulted to 2,048 tokens, truncating multi-unit outputs mid-JSON. Remediated by setting `PARAMETER num_ctx 8192` and `PARAMETER num_predict 2048`.

2) Silent Fallback: `intent_router.py` required `USE_LLM_ROUTER=true` which was missing in `.env`, silently defaulting to cosine routing. Remediated by updating configuration and extending LLM timeout to 10.0 s.

3) Multimodal Leakage: Visual extraction functions were feeding base64 payloads to campus-llm. Remediated by enforcing strict delegation to `OLLAMA_VISION_MODEL = gemma4:12b-it-qat`.

4) Vision Configuration: Pydantic settings in `config.py` lacked vision model properties. Remediated by adding `ollama_vision_model: str = "gemma4:12b-it-qat"`.

XI. DISCUSSION

A. Significance of Architectural Decoupling

The 16.3× latency reduction proves the value of separating high-velocity textual reasoning from visual processing. Attempting to deploy a single monolithic model introduces memory bloat and latency bottlenecks unacceptable on institutional hardware.

B. Response-Only Loss Masking

Masking user prompt tokens during training ensures the SLM optimizes exclusively for generation of structured assistant responses, avoiding loss distortion from prompt instructions.

C. JSON Compliance as a Hard Constraint

In automated agentic systems, JSON compliance is an absolute operational requirement. Improving compliance from 84.2% to 99.8% eliminates downstream parsing crashes.

D. Limitations

Limitations include: (1) single-pass static RAGAS evaluation, (2) regex edge cases on typographic variations in textbook headings, (3) lack of cross-database distributed transactions across Neo4j and PostgreSQL, and (4) desynchronized unit test constants.

XII. CONCLUSION

We demonstrated M.A.C.H., an academic knowledge engine combining a fine-tuned 3B Small Language Model (campus-llm) with a hybrid Graph-RAG architecture. By decoupling text reasoning from visual extraction, training over 9,294 multi-task instances with QLoRA, and fusing Neo4j KG traversal with pgvector HNSW search, M.A.C.H. achieves 99.8% schema compliance, 16.3× latency speedup, and robust 0.91 RAGAS faithfulness—delivering an edge-deployable, high-precision campus cognitive hub.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *NeurIPS*, vol. 33, pp. 9459–9474, 2020.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.

[3] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in *ICLR*, 2022.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *NeurIPS*, vol. 36, pp. 10088–10115, 2023.

[6] Qwen Team, Alibaba Group, "Qwen2.5 Technical Report," *arXiv preprint arXiv:2412.15115*, 2024.

[7] R. Guo, Q. Zhao, Q. Chen, L. Tang, and T. Chen, "A Knowledge Graph-Based Approach to Personalized Learning Path Recommendation," *IEEE Trans. Learning Technol.*, vol. 15, no. 4, pp. 504–517, 2022.

[8] C. Liang, J. Wu, T. Xiong, W. Zhang, Y. Hu, and H. Wang, "Recovering Concept Prerequisite Relations from University Course Dependencies," in *Proc. AAAI*, vol. 32, 2018.

[9] X. Hu, P. Ng, J. Ma, Z. Li, and B. Xiang, "MESE: Modelling Educational Skill Entities from Behavioral Sequences," in *Proc. ACM SIGKDD*, pp. 699–709, 2022.

[10] B. S. Bloom, M. D. Engelhart, E. J. Furst, W. H. Hill, and D. R. Krathwohl, *Taxonomy of Educational Objectives: The Classification of Educational Goals, Handbook I: Cognitive Domain*. New York: David McKay Co., 1956.

[11] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in *Proc. ACM SIGIR*, pp. 758–759, 2009.

[12] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[13] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," in *NeurIPS Datasets and Benchmarks*, 2021.

[14] Z. Shen, K. Zhang, M. Dell, B. C. G. Lee, J. Carlson, and W. Li, "LayoutParser: A Unified Toolkit for Deep Learning Based Document Image Analysis," in *Proc. ICDAR*, pp. 131–148, 2021.

[15] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *Proc. EACL*, pp. 150–163, 2024.

[16] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[17] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proc. EMNLP Demos*, pp. 38–45, 2020.

[18] Y. Malkov and D. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE TPAMI*, vol. 42, no. 4, pp. 824–836, 2020.

[19] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[20] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, "TriviaQA: A Reading Comprehension Dataset for Evaluation," in *Proc. ACL*, pp. 1601–1611, 2017.

APPENDIX A: CODEBASE ARCHITECTURE & DATABASE SCHEMAS

TABLE IX: CORE BACKEND MODULES IN M.A.C.H. PRODUCTION REPOSITORY

Module Name	Source Location	Primary Functional Responsibility
Application Core	`backend/app.py`	FastAPI routing, lifespan startup/shutdown, chat endpoint, SSE token streaming
Curriculum Parser	`backend/curriculum_extractor.py`	Regex + LLM boundary extraction, syllabus normalization, Neo4j graph materializer
Database Layer	`backend/database.py`	AsyncPG connection pooling, Neo4j GraphDatabase driver singleton instance
Embedding Engine	`backend/embedder.py`	Ollama nomic-embed-text wrapper, in-memory LRU(2000) cosine embedding cache
Intent Router	`backend/intent_router.py`	8-class cosine + LLM intent classification, fallback router
PYQ Processor	`backend/pyq_processor.py`	Question persistence, duplicate hash check, pre-flight guard, report generator
Graph Queries	`backend/query_neo4j.py`	Optimized Cypher query builders for Graph-RAG context & Canvas visualization
Topic Mapper	`backend/topic_mapper.py`	Two-stage dense + LLM question-to-topic mapping with confidence calibration
Utilities Engine	`backend/utils.py`	PyMuPDF visual slicer, Cloudinary upload, RRF SQL execution, query rewriter
Vision Queries	`backend/vision_query.py`	Multimodal reverse lookup for student-uploaded examination diagrams

Neo4j Constraints & Index DDL:

```
CREATE CONSTRAINT doc_id_unique IF NOT EXISTS
  FOR (d:Document) REQUIRE d.id IS UNIQUE;
CREATE CONSTRAINT question_id_unique IF NOT EXISTS
  FOR (q:Question) REQUIRE q.id IS UNIQUE;
CREATE CONSTRAINT topic_id_unique IF NOT EXISTS
  FOR (t:Topic) REQUIRE t.id IS UNIQUE;
CREATE CONSTRAINT course_code_unique IF NOT EXISTS
  FOR (c:Course) REQUIRE c.code IS UNIQUE;
CREATE INDEX topic_normalized_name_idx IF NOT EXISTS
  FOR (t:Topic) ON (t.normalized_name);
CREATE INDEX question_course_idx IF NOT EXISTS
  FOR (q:Question) ON (q.course_code);
CREATE INDEX topic_approved_idx IF NOT EXISTS
  FOR (t:Topic) ON (t.approved);
```

PostgreSQL 16 & pgvector DDL:

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    filename VARCHAR(255) NOT NULL,
    department VARCHAR(50), course_code VARCHAR(20),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    processing_status VARCHAR(20) DEFAULT 'pending'
);

CREATE TABLE document_chunks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id UUID REFERENCES documents(id) ON DELETE CASCADE,
    content TEXT NOT NULL, course_code VARCHAR(20),
    embedding VECTOR(768),
    tsv_content TSVECTOR GENERATED ALWAYS AS
        (to_tsvector('english', content)) STORED
);

CREATE INDEX ix_document_chunks_embedding_hnsw
  ON document_chunks USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
CREATE    INDEX    idx_chunks_tsv    ON    document_chunks    USING
GIN(tsv_content);
```